

ATM MACHINE SIMULATION SYSTEM

PROJECT REPORT

Submitted as part of the VITyarthi – Build Your Own Project

Submitted By

Somya Khandelwal

Registration Number: 26MIP10091

Course: Integrated M.Tech in Computational and Data Science

VIT Bhopal University

Academic Year: 2026

GitHub Repository

<https://github.com/somyakhandelwal2207/atm-machine>

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Functional Requirements
- 5. Non-Functional Requirements
- 6. System Architecture
- 7. Design Diagrams
- 8. Design Decisions & Rationale
- 9. Implementation Details
- 10. Screenshots / Results
- 11. Testing Approach
- 12. Challenges Faced
- 13. Learnings & Key Takeaways
- 14. Future Enhancements
- 15. References

1. Introduction

The ATM Machine Simulation System is a Python-based command-line application that demonstrates the basic operations of an Automated Teller Machine in a software environment. The project was developed to apply programming concepts through a practical simulation of common banking activities.

The system provides account creation and authentication, followed by a menu through which a user can check balance, deposit money, withdraw money, transfer money, view account details, view transaction history, change the PIN, and log out.

The project uses a modular Python structure. Separate files are used for authentication, account management, transaction processing, transaction history, input validation, and account data. The system stores account information in an in-memory Python dictionary for the purpose of the simulation.

2. Problem Statement

Traditional ATM systems provide several banking operations through a controlled user interface. For learning purposes, it is useful to model these operations in a small software system so that the flow of authentication, account management, transactions, validation, and data updates can be understood.

The problem addressed by this project is to design and implement a simple ATM simulation that demonstrates these core operations while maintaining a clear logical workflow and modular code structure.

3. Objectives

- Develop a simple ATM simulation using Python.
- Implement account creation and user authentication.
- Provide balance checking, deposit, withdrawal, and money transfer operations.
- Maintain transaction history for the simulated accounts.
- Provide account details and PIN-change functionality.
- Apply input validation and basic error handling.
- Organize the implementation into meaningful Python modules.
- Test the major ATM operations through functional execution and validation tests.

4. Functional Requirements

The system provides the following functional modules and capabilities:

Module	Requirement
Account Management	Create a new account, view account details, and change the account PIN.
Authentication	Allow login using an account number and PIN and reject invalid credentials.
Transaction Management	Check balance, deposit money, withdraw money, and transfer money between accounts.
Transaction History	Record and display deposit, withdrawal, and transfer activities.
Input Validation	Validate transaction amounts and provide error messages for invalid input.

5. Non-Functional Requirements

- Usability – the command-line menu should be simple and easy to understand.
- Security – authentication requires an account number and PIN before ATM operations are accessed.
- Reliability – valid transactions should update the simulated account balance consistently.
- Performance – the local simulation should respond to menu selections and transactions without unnecessary delay.
- Maintainability – the project is divided into separate Python modules so individual functions can be maintained more easily.
- Error Handling – invalid choices, invalid amounts, insufficient balance, and invalid account numbers should be handled with appropriate messages.

6. System Architecture

The ATM Machine Simulation System follows a modular architecture. The main program controls the overall application flow and connects the ATM menu with authentication, account management, transaction processing, transaction history, validation, and account data.

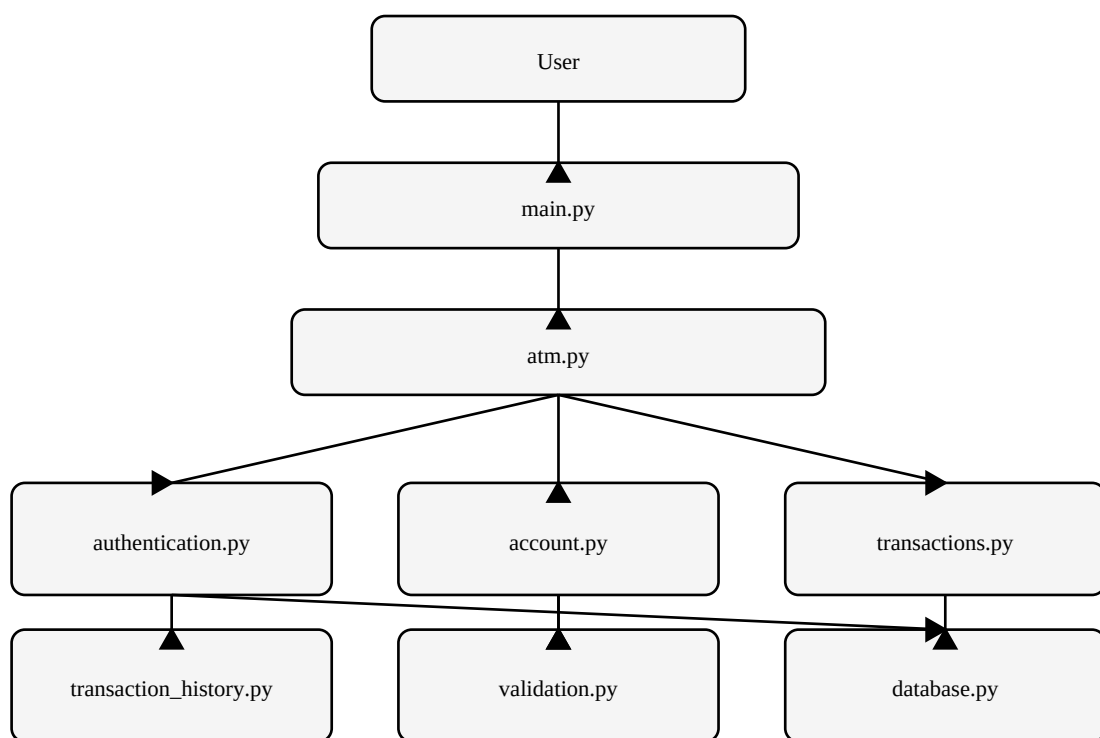


Figure 1. System Architecture Diagram

7. Design Diagrams

The following diagrams describe the workflow and interactions of the ATM system.

7.1 Workflow Diagram

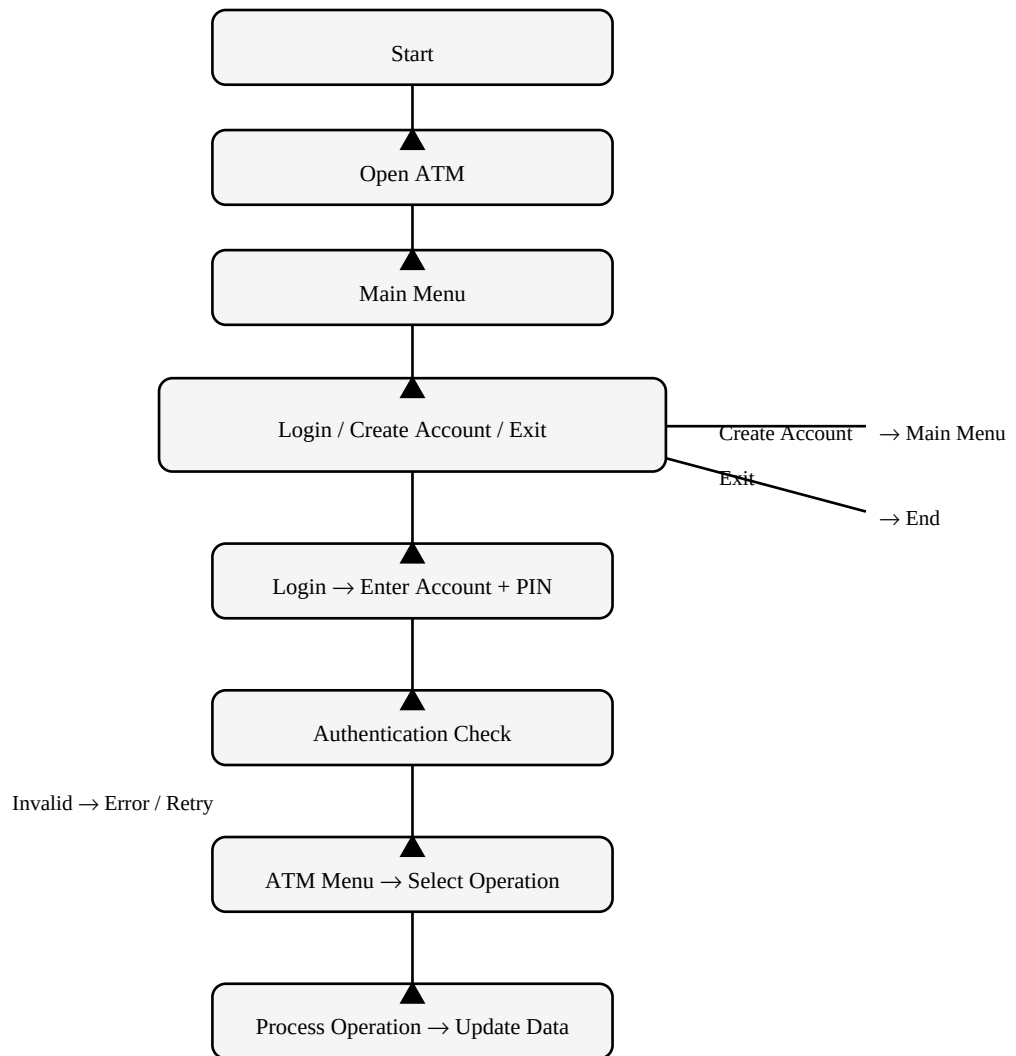


Figure 2. Workflow Diagram

The workflow begins at the main ATM menu. The user can log in, create an account, or exit. After successful authentication, the user enters the ATM menu, selects an operation, and the system updates the relevant account data before allowing the user to continue or log out.

7.2 Use Case Diagram

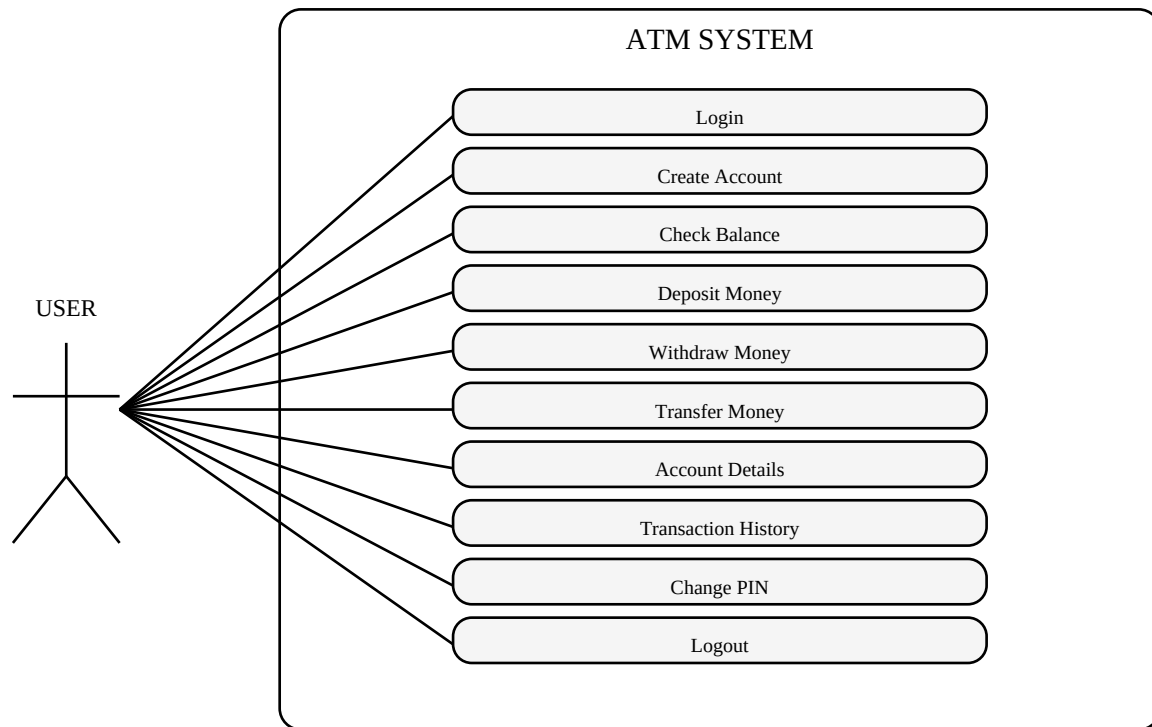


Figure 3. Use Case Diagram

The Use Case Diagram represents the interaction between the user and the ATM system. The user can access account and transaction operations after entering the system.

7.3 Class / Component Diagram

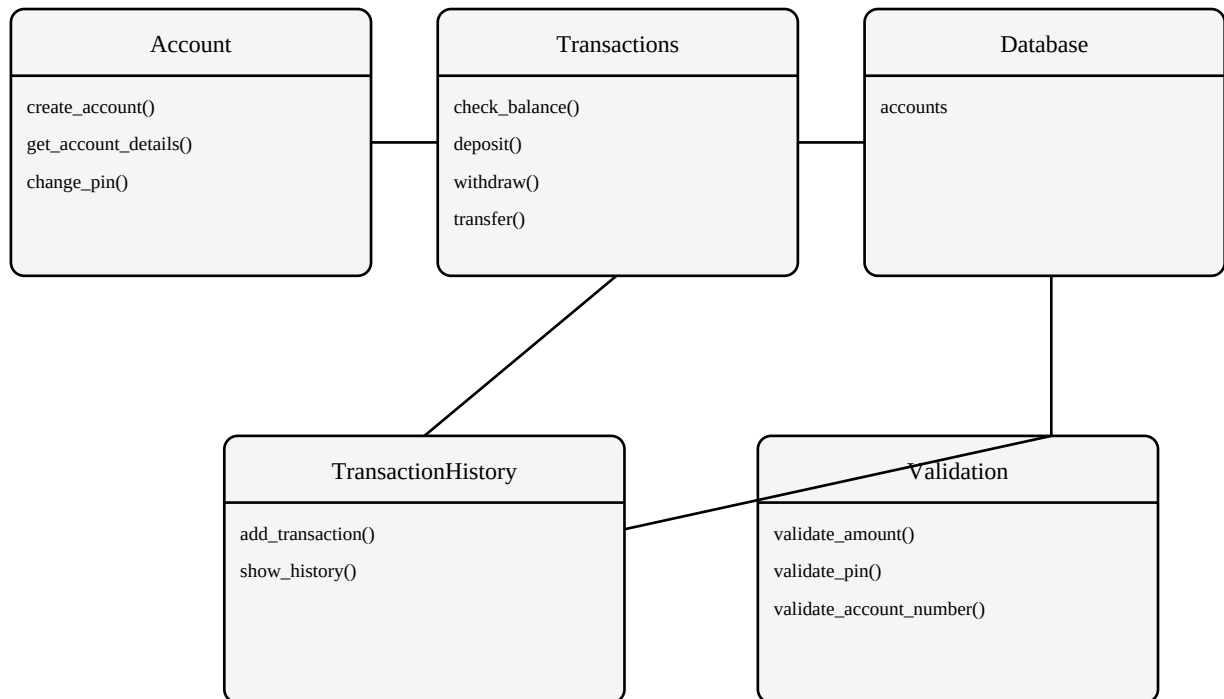


Figure 4. Class / Component Diagram

The diagram represents the main modules used by the project: account management, transaction processing, account data storage, transaction history, and validation.

7.4 Sequence Diagram

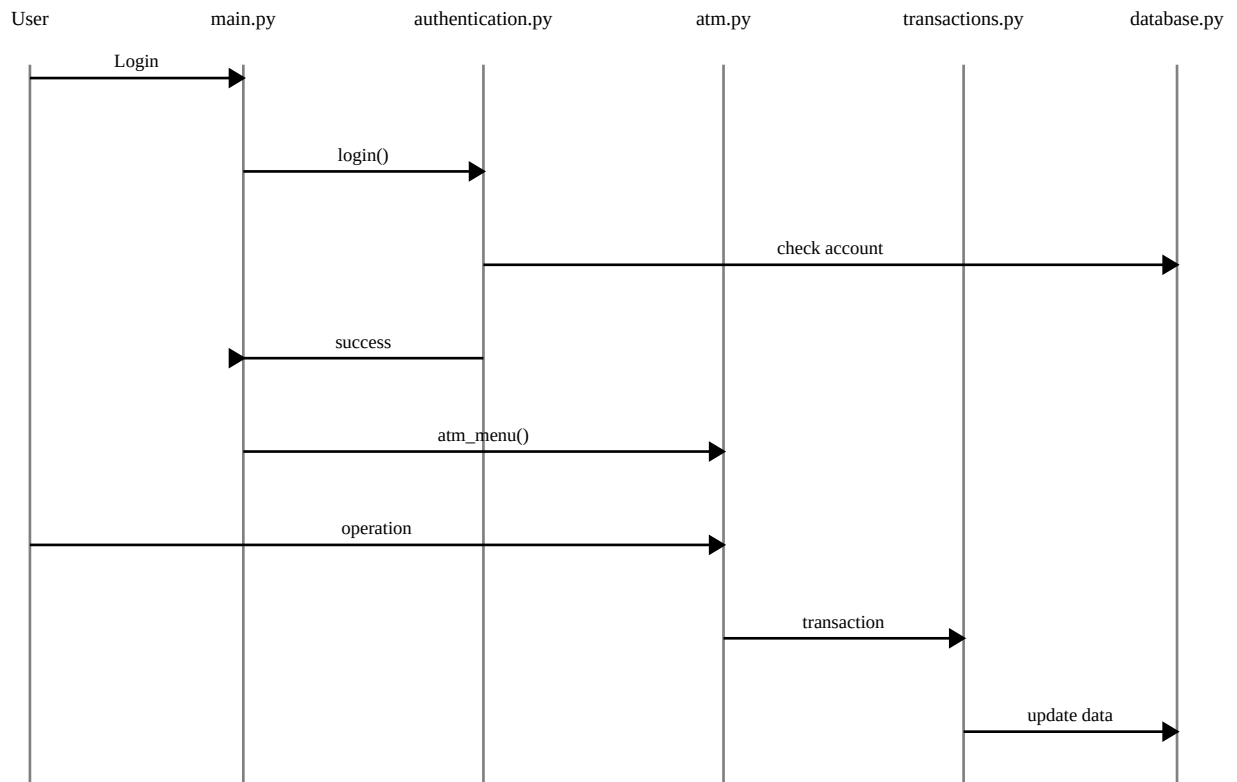


Figure 5. Sequence Diagram

The sequence illustrates a typical interaction in which the user logs in, authentication checks the account data, the ATM menu is opened, an operation is selected, and the transaction module updates the account data.

7.5 Storage / Data Structure Design

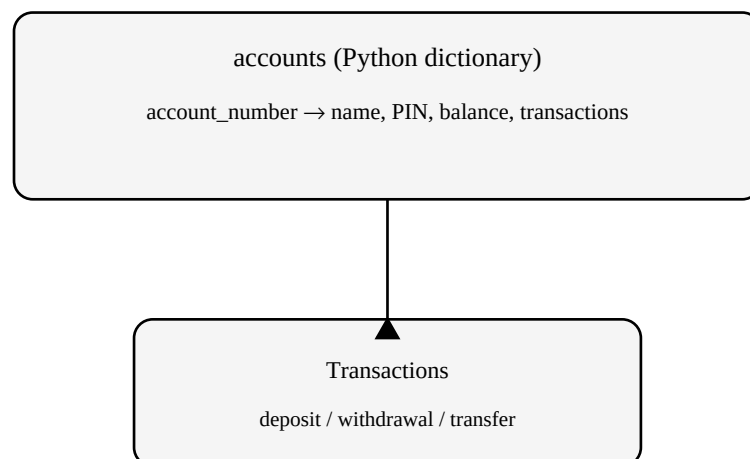


Figure 6. Storage / Data Structure Diagram

The project does not use an external SQL database. Account information is stored in the Python dictionary defined in database.py. Each account record contains a name, PIN, balance, and transaction list.

8. Design Decisions & Rationale

- Python was selected because it supports simple procedural and modular programming and is suitable for demonstrating the logic of an ATM simulation.
- A command-line interface was used to keep the project focused on the core programming logic rather than graphical interface development.
- The project is divided into multiple Python files so that authentication, account management, transactions, validation, and history are separated.
- An in-memory dictionary was used for account storage because the project is a simulation and does not require an external database.
- Transaction history is stored as a list associated with each account so that completed operations can be displayed during the session.

9. Implementation Details

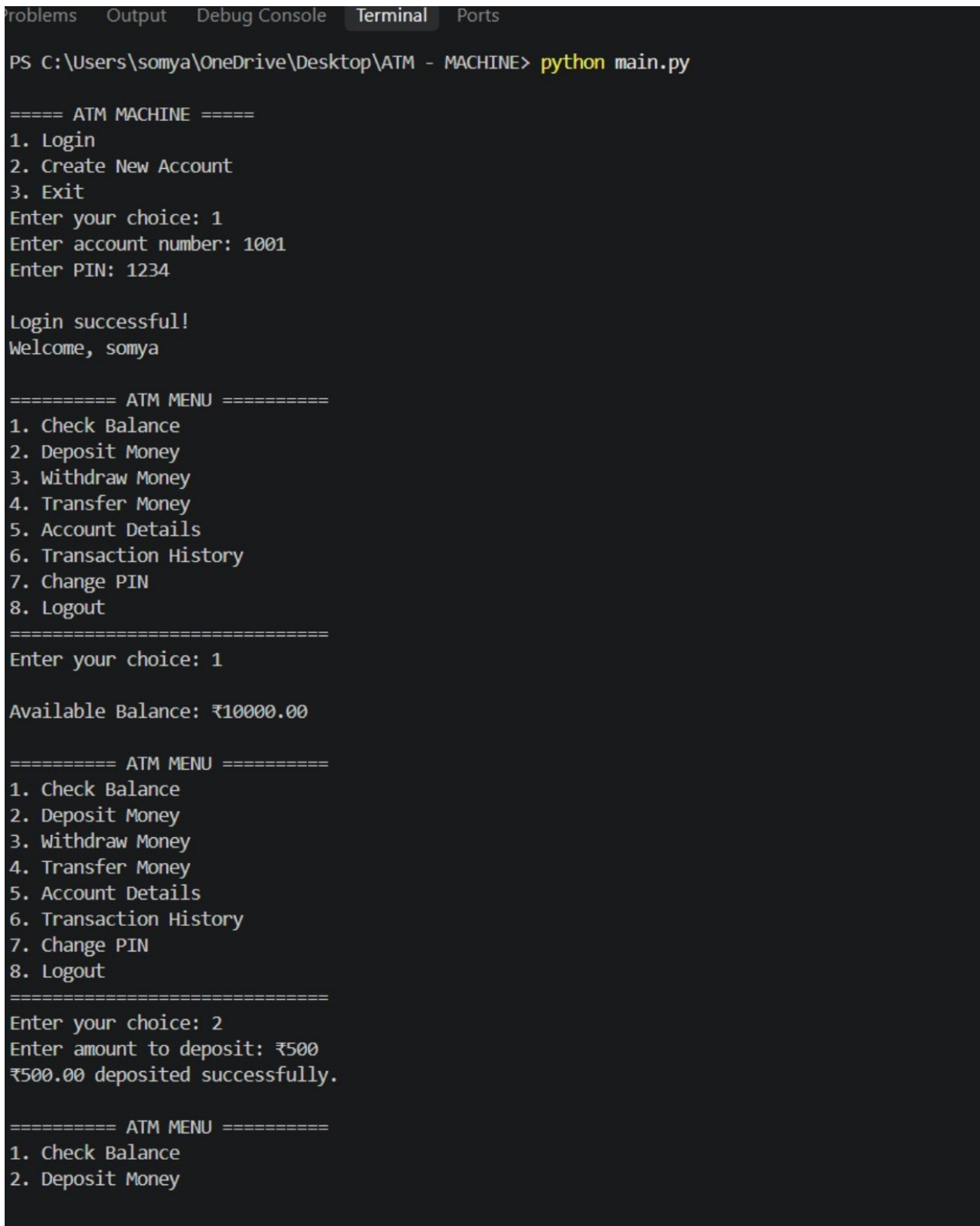
The implementation is organized into separate Python modules. The main files and their responsibilities are listed below:

File	Purpose
main.py	Starts the application and handles the main menu.
atm.py	Displays the ATM menu and routes user selections.
authentication.py	Performs account number and PIN verification.
account.py	Handles account creation, account details, and PIN changes.
transactions.py	Handles balance checking, deposits, withdrawals, and transfers.
transaction_history.py	Displays transaction history.
validation.py	Provides validation functions for amounts, PINs, and account numbers.
database.py	Stores the simulated account data in a Python dictionary.

The main program uses a loop to present the main menu. A successful login passes the account number to the ATM menu, where each selection calls the corresponding function. Transaction functions update the balance and add transaction records to the account's transaction list.

10. Screenshots / Results

The following screenshots show the working command-line output of the ATM Machine Simulation System. They demonstrate login, balance checking, deposit, withdrawal, transfer, account details, transaction history, PIN change, logout, account creation, and exit operations.

A screenshot of a Windows terminal window with tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The command prompt shows the user is in the directory 'C:\Users\somya\OneDrive\Desktop\ATM' and has executed 'python main.py'. The program output is as follows:

```
PS C:\Users\somya\OneDrive\Desktop\ATM - MACHINE> python main.py

===== ATM MACHINE =====
1. Login
2. Create New Account
3. Exit
Enter your choice: 1
Enter account number: 1001
Enter PIN: 1234

Login successful!
Welcome, somya

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 1

Available Balance: ₹10000.00

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 2
Enter amount to deposit: ₹500
₹500.00 deposited successfully.

===== ATM MENU =====
1. Check Balance
2. Deposit Money
```

Figure 7. Login, balance checking, deposit, and ATM menu output.

```
Problems Output Debug Console Terminal Ports

=====
Enter your choice: 2
Enter amount to deposit: ₹500
₹500.00 deposited successfully.

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 3
Enter amount to withdraw: ₹398
Please collect your cash: ₹398.00

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 4
Enter receiver account number: 1002
Enter amount to transfer: ₹60
Money transferred successfully.

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
```

Figure 8. Withdrawal and money transfer output.

```
Problems Output Debug Console Terminal Ports

=====
Enter your choice: 5

Name: somya
Account Number: 1001
Balance: ₹ 10042.0

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 6

===== TRANSACTION HISTORY =====
1. Deposited ₹500.00
2. Withdrawn ₹398.00
3. Transferred ₹60.00 to 1002

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 7
Enter new PIN: 4444
PIN changed successfully!

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
```

Figure 9. Account details, transaction history, and PIN change output.

```
Problems Output Debug Console Terminal Ports

2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 7
Enter new PIN: 4444
PIN changed successfully!

===== ATM MENU =====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Account Details
6. Transaction History
7. Change PIN
8. Logout
=====
Enter your choice: 8

Thank you for using the ATM!

===== ATM MACHINE =====
1. Login
2. Create New Account
3. Exit
Enter your choice: █
```

Figure 10. Logout and return to the main ATM menu.

=====

Enter your choice: 8

Thank you for using the ATM!

===== ATM MACHINE =====

1. Login

2. Create New Account

3. Exit

Enter your choice: 2

Enter your name: shivi

Create PIN: 6666

Enter initial deposit: 900

Account created successfully!

Your account number is: 1003

===== ATM MACHINE =====

1. Login

2. Create New Account

3. Exit

Enter your choice: 3

Thank you for using ATM!

PS C:\Users\somya\OneDrive\Desktop\ATM - MACHINE> █

Figure 11. New account creation and program exit output.

11. Testing Approach

The ATM system was tested by running the application in the terminal and exercising the major functions. The observed output demonstrates successful execution of the main user flows.

- Login using a valid account number and PIN.
- Check the available balance.
- Deposit money and verify the success message.
- Withdraw money and verify the cash output.
- Transfer money to another account.
- View account details.
- View transaction history and verify recorded transactions.
- Change the PIN and verify the confirmation message.
- Log out and return to the main menu.
- Create a new account and verify that a new account number is generated.
- Exit the application.

Validation test scripts are also included in the project for login, deposit, and withdrawal functionality. The project was tested through actual terminal execution, and the screenshots in Section 10 provide evidence of the tested flows.

12. Challenges Faced

- Organizing the ATM application into multiple modules while keeping the functions connected correctly.
- Managing account data and transaction history using an in-memory dictionary and lists.
- Handling invalid transaction amounts and insufficient balance conditions.
- Maintaining a clear menu flow between the main menu, login, ATM operations, and logout.
- Testing multiple transaction operations and checking that the resulting balance and history were updated.

13. Learnings & Key Takeaways

- Learned how to divide a Python project into multiple modules.
- Understood how functions can communicate through imported modules.
- Practiced using dictionaries and lists to represent structured account data.
- Learned how authentication and validation can be added to a command-line application.
- Practiced implementing transaction logic such as deposits, withdrawals, and transfers.
- Learned the importance of testing each major feature after implementation.
- Understood how project documentation and design diagrams help explain software structure.

14. Future Enhancements

- Store account data permanently using a database such as SQLite or MySQL.
- Improve PIN security by storing protected credentials instead of plain text PINs.
- Add stronger input validation for account creation and PIN changes.
- Add detailed transaction reports with dates and times.
- Develop a graphical user interface.
- Add stronger authentication and session controls.
- Support more advanced banking features in a larger version of the system.

15. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
2. Python documentation and standard language concepts used during implementation.
3. Project source code and execution output produced during development.

GitHub Repository

GitHub Repository: <https://github.com/somyakhandelwal2207/atm-machine>

The complete project source code is available in the GitHub repository above.